QuantumTrace: perceber a pasta e defender os resultados

Guia em português da distribuição 0.3.3. Hidden Signal. JunctionX Lisbon 2026. 10 de outubro de 2026.

Este guia cobre os 97 ficheiros do ZIP atual. Não verifica se a tua instalação Windows tem exatamente estes bytes. Os caminhos são relativos à pasta exterior quantumtrace, onde está INICIAR_APP.cmd. O PowerPoint e o roteiro são materiais separados. A revisão final também melhorou as prioridades e a política visível da app, conservando as classificações e os dados originais.

1. O raciocínio em dois minutos

Uma organização não consegue planear uma migração criptográfica se não souber onde usa criptografia. O QuantumTrace recolhe indícios de código, configurações e capturas TLS, transforma-os num inventário comum e aplica regras documentadas a cada primitiva. O resultado liga a classificação à origem e sugere um próximo passo.

Uma primitiva é um mecanismo como AES, RSA ou SHA-256. Uma observação é uma ocorrência desse mecanismo num local do código, num campo JSON ou num evento TLS. Um ficheiro pode conter várias observações. Uma suite pode decompor-se em várias primitivas. Nada disto equivale automaticamente a um número de aplicações, recursos ou vulnerabilidades únicas.

A lógica tem quatro responsabilidades: receber dados, reconhecer padrões, avaliar as primitivas reconhecidas e mostrar a evidência. Separámos essas responsabilidades para poderes rever uma classificação sem modificar os dados e melhorar o detetor sem reconstruir a interface.

Para começar, lê as secções 1 a 6. Depois abre inventory.py, scanners.py, static_facts.py, policy.py, report.py e app.py por esta ordem. As secções 7 a 20 são um catálogo para consulta, com todos os ficheiros. Termina pela secção 21 e ensaia com o roteiro em inglês.

2. Quem escreveu o quê e como evoluiu

O código do motor, da app, dos testes, dos relatórios-modelo e os exemplos próprios foram produzidos com assistência de IA nesta conversa, seguindo os teus pedidos e correções. Não deves dizer que escreveste manualmente todos estes ficheiros. A tua contribuição inclui a direção do projeto, escolhas, testes no PC, avaliação da interface e a responsabilidade de perceber e apresentar a solução.

Os 35 ficheiros de data são material sintético fornecido pela organização através do repositório indicado. Foram conservados sem alteração. Python, SQLite, TShark e cryptography são tecnologias externas; não escrevemos essas bibliotecas. Os relatórios preenchidos são produzidos automaticamente pelo motor, não números inventados manualmente.

Etapa	O que mudou	Motivo
Primeira versão	Inventário comum, detetores iniciais, política e relatório	Ter um percurso completo de dados até evidência
0.2	Mais padrões de código, parâmetros, AWS e captura	Melhorar a cobertura dos exemplos fornecidos
0.3	Política contextual de hashes e revisão, app local, API, SQLite e comparação	Explicar casos difíceis e permitir analisar novos dados
0.3.1	Correções de links e tratamento do token visível	Melhorar consulta das fontes e experiência na app
0.3.2	Eliminação individual e limpeza do histórico	Dar controlo sobre os relatórios guardados
0.3.3	Prioridades por ficheiro/recurso, recomendações de híbridos e política simples	Ajudar a decidir o próximo passo e congelar a versão

Na passagem de 0.2 para 0.3, os dados não mudaram. Os desconhecidos dos cartões passaram de 118 para 9 pela combinação de contexto e regras novas. Isso não significa que 109 vulnerabilidades tenham sido corrigidas. Uma mudança de política também pode mudar o índice.

3. O que acontece quando carregas em analisar

Passo	Ficheiros principais	Raciocínio
Escolha	index.html e app.js	O utilizador escolhe os dados do desafio ou ficheiros/pasta
Receção	app.py	Recebe JSON local com ficheiros em base64, valida nomes/limites e cria uma tarefa
Percurso	inventory.py	Lê os bytes, calcula SHA-256 e escolhe o detetor pelo formato
Deteção	scanners.py, static_facts.py, source_patterns.py, cloud.py e tls.py	Recolhe os mecanismos e os parâmetros suportados
Avaliação	policy.py e Collector	Aplica uma regra ao algoritmo, parâmetros, função e contexto
Resultado	inventory.py e report.py	Calcula o resumo e constrói JSON/HTML
Consulta	app.py, app.js e report.js	Guarda o relatório no histórico e mostra detalhes, filtros e exportação

A app recebe novos dados pelo seletor de ficheiros ou de pasta. Não precisas de editar INICIAR_APP.cmd nem de chamar uma API paga. A API local recebe o conteúdo e processa-o neste computador. Os uploads brutos são temporários; o SQLite guarda relatórios. O código de entrada é lido, não executado.

Podes escolher um projeto completo, respeitando 2000 ficheiros, 20 MiB por ficheiro e 80 MiB por seleção na app. O scanner ignora pastas como .git, ambientes Python, node_modules, caches, reports e o estado da app. ZIP precisa de extração. Isto não torna suportadas todas as linguagens, binários ou bibliotecas desse projeto.

Abrir histórico ou report.html só consulta um resultado anterior. Uma nova análise volta a ler os inputs. Com os mesmos bytes, ferramentas e política, as observações normalmente repetem-se; data e duração podem mudar. A comparação distingue alterações de inputs, cobertura e política, e não afirma que uma observação removida foi corrigida.

O ficheiro .quantumtrace/history.sqlite3 surge em execução e não está nos 97 ficheiros do ZIP. A app cria-o e guarda o JSON de cada relatório, nome, data e identificador. Apagar histórico remove esses relatórios, não os originais nem exports já descarregados.

4. Como reconhece criptografia e como lês uma evidência

O motor não procura a palavra criptográfico. Em Python reconhece chamadas de APIs suportadas e resolve imports/aliases usando uma árvore sintática, a representação estrutural do código. Em Java/Go usa padrões de chamadas e inferência limitada por função. Em JSON procura campos conhecidos, como Algorithm ou KeySpec. Em TLS interpreta campos específicos do handshake.

Exemplo de 01_aes128.py:

```
from secrets import token_bytes
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

chave = token_bytes(16)
cifra = AESGCM(chave)
```

scanners.py reconhece AESGCM através do import. static_facts.py segue chave até token_bytes(16) e conserva apenas o comprimento. 16 vezes 8 dá 128 bits. policy.py classifica AES-128 como médio segundo o desafio. Collector guarda a linha 6, o detetor e o hash do ficheiro. O programa não gerou a chave nem cifrou dados para produzir esta observação.

Campo visível	O que quer dizer
Primitive e Key bits	Algoritmo identificado e parâmetro conhecido. ML-KEM-768 não é uma chave de 768 bits
Tier	Nível atribuído pela política da versão
Source / locator	Ficheiro e linha, ponteiro JSON ou frame da captura onde apareceu
Evidence	Indício concreto reconhecido pelo detetor
Mechanism	Motivo quântico associado ou unresolved quando falta avaliação
Classification e Rule	Explicação e identificador da regra aplicada
Next step	Sugestão de investigação ou migração, sem reescrever o projeto
Missing information	O que falta para resolver a observação por rever
Evidence confidence	Confiança qualitativa na evidência. medium aqui não é o nível de risco nem uma percentagem de precisão
Role e Context	Função do mecanismo e dados como operação, modo, sessão ou utilização do hash
Model e Assumptions	Modelo usado, quando existe, e as hipóteses de avaliação
Standards e Source documentation	Normas atribuídas pela regra e links de referência. Not assigned não significa que AES não tenha uma norma

Campo visível	O que quer dizer
Classical concern e Caveats	Avisos específicos e limites. Ausência de aviso não prova ausência de falhas

O SHA-256 do ficheiro serve para reconhecer os mesmos bytes e apoiar comparações. Não prova sozinho que o scanner interpretou corretamente esses bytes. A regra e o local de origem é que tornam a decisão inspecionável.

Tipo de observação	O que demonstra
configured	Um campo de configuração declara o mecanismo ou permite uma capacidade
static-call	O código contém uma chamada reconhecida, sem provar execução real
offered	O cliente anunciou uma opção TLS, sem provar seleção
negotiated	Um evento de handshake observado seleciona o mecanismo suportado
key-material	Uma chave pública foi interpretada pelo parser opcional
certificate-signature	Identificação da assinatura do emissor, distinta da chave pública do certificado

5. Contagens e Primitive exposure index

No conjunto da organização: 35 entradas, 28 ficheiros analisados e sete metadados não suportados. São 1574 observações: 619 nos cartões e 955 ofertas TLS. Os 619 incluem 238 críticas, 274 médias, 98 resilientes e nove por rever. Não são 1574 ficheiros.

O Primitive exposure index é uma média com pesos definidos pelo projeto: crítico vale 100, médio 50 e resiliente 0. Só entram observações classificadas e não oferecidas. Os desconhecidos ficam fora da soma e do denominador, sempre apresentados em separado.

```
Dados do desafio:
(238 x 100 + 274 x 50 + 98 x 0) / (238 + 274 + 98)
= 37500 / 610 = 61,4754... = 61,5 / 100

Oito exemplos novos:
(2 x 100 + 2 x 50 + 4 x 0) / (2 + 2 + 4)
= 300 / 8 = 37,5 / 100
```

O índice não é uma probabilidade de ataque nem uma percentagem de sistemas inseguros. A quantidade de componentes e repetições altera a média. Um índice zero também não certifica segurança. Sem observações classificadas, o índice é indisponível, não zero.

Offered mechanisms are excluded from the index quer dizer que as opções anunciadas no ClientHello ficam no inventário, mas fora desta média e dos cartões. Não é correto tratá-las

como mecanismos efetivamente escolhidos. Mesmo as observações dos cartões têm contextos diferentes: configuração e chamada estática também não provam utilização em execução.

98,5% é 610 / 619 vezes 100, arredondado. Chama-se cobertura de classificação: quantas observações dos cartões receberam nível. Não mede quantas criptografias reais foram descobertas corretamente. Um detetor pode ter alta cobertura de classificação e continuar a falhar padrões que nunca reconheceu.

6. Política, casos por rever e os quatro tópicos

Tópico do desafio	Implementação atual	Limite
1. Network discovery	Capturas e JSON TLS com ofertas, escolhas e grupos	Campos TLS suportados, sem auditoria de todo o tráfego
2. Application and code	Python/Java/Go, JSON e chaves públicas/certificados opcionais	Padrões limitados e CBOM próprio. Dependências não são inventariadas de forma completa
3. Classification	Regras explícitas de primitivas e contexto	Classifica componentes, sem certificar o sistema
4. Dashboard/reporting	CLI, app local, evidência, índice, histórico e JSON/HTML	CycloneDX e modelo empresarial calibrado ainda não implementados

O enunciado permite escolher um ou mais tópicos. Este protótipo cobre partes dos quatro, com profundidades diferentes. Os dois primeiros recolhem a evidência necessária para o terceiro. Não é obrigatório implementar os quatro para cumprir aquela frase do enunciado; também não devemos dizer que cobrimos tudo o que cada tópico permite.

Os níveis básicos seguem o desafio: famílias clássicas de chave pública recebem crítico pela exposição a Shor; AES-128/192 recebe médio; AES-256 e famílias PQC finais reconhecidas recebem resiliente. Isto é avaliação de primitivas. A categoria AES-128 do desafio não afirma um ataque prático atual. A FAQ NIST explica limites práticos do ganho de Grover e mantém orientação de utilização para AES com os tamanhos existentes.

A extensão de hashes é nossa: usa um alvo de 128 bits num modelo ideal de consultas quânticas. Para colisões usa n/3; para pré-imagem n/2, sendo n o tamanho do digest. SHA-256 pode receber níveis diferentes segundo o contexto. A regra TLS escolhe um modelo conservador para a componente hash, sem provar uma avaliação de todos os usos HKDF/MAC. Não apresenta estes expoentes como segundos de ataque nem categorias NIST.

Os nove casos ainda por rever são cinco HMAC-SHA-256, três PBKDF2-HMAC-SHA-256 e uma componente SHA-256 dentro de RSA-OAEP. Nos oito primeiros faltam garantias sobre segredo, palavra-passe ou parâmetros. OAEP é uma construção identificada cuja regra específica ainda falta. A componente RSA continua a surgir como crítica.

No exemplo 04, AES e GCM são reconhecidos, mas o comprimento da chave não aparece. unresolved refere-se à avaliação pendente, não a desconhecer o nome AES. O próximo passo é

seguir a origem da chave ou obter configuração/chamadores que estabeleçam 128, 192 ou 256 bits.

Um grupo X25519MLKEM768 contém um componente clássico e um PQC. O programa conserva ambos. Classificar X25519 como crítico não demonstra que o mecanismo híbrido combinado esteja quebrado. A segurança da composição, autenticação e implementação exige outra avaliação.

Na versão 0.3.3, a área de prioridades agrupa as observações por ficheiro, recurso AWS ou sessão TLS. Cada grupo mostra mecanismos, estado da evidência, motivo e próximo passo, com acesso às observações. Para chave pública, uma seleção observada recebe revisão antes de uma configuração e de uma chamada estática. Ofertas são apenas capacidades inventariadas; não geram uma prioridade de migração. Híbridos recebem revisão da construção completa, sem recomendar substituição apenas por terem uma componente clássica crítica.

No conjunto do desafio há 137 grupos: 25 de ficheiro, 15 de recurso AWS e 97 de sessão TLS. Não são 137 ativos empresariais únicos. Sem responsáveis, impacto ou tempo de confidencialidade, a ordem é técnica e não um ranking completo de risco. Os níveis dos hashes mantêm a extensão documentada do projeto. O índice continua 61,5. A versão foi congelada num novo ZIP para ensaio; a cópia 0.3.2 foi preservada.

7. Ficheiros da raiz: abrir, executar e distribuir

A raiz é a pasta exterior chamada quantumtrace. A subpasta quantumtrace contém o programa Python. Os atalhos Windows ficam na raiz para encontrarem o programa e os dados através de caminhos relativos.

INICIAR_APP.cmd

Abre a aplicação no navegador e mantém o servidor Python a funcionar. Verifica se a pasta foi extraída e se existe Python 3.10 ou superior. Não fixa o ficheiro a analisar: escolhes os dados na interface. A janela do terminal deve ficar aberta.

ABRIR_RELATORIO.cmd

Abre reports/report.html já existente. Não volta a analisar os dados e não atualiza os testes. Serve para consultar um resultado guardado, incluindo quando precisas de uma alternativa offline.

ANALISAR_DADOS.cmd

Verifica Python e ferramentas opcionais, chama run_demo.py e abre o resultado. Este atalho volta a analisar a pasta data e substitui os dois relatórios em reports. Não é necessário editar este atalho para selecionar outros ficheiros na app.

run_demo.py

Localiza a raiz a partir do próprio ficheiro e chama o comando scan sobre data, com saída em reports. Isto evita depender da pasta onde abriste o terminal. O nome demo refere-se aos dados sintéticos do desafio incluídos nesta distribuição.

pyproject.toml

Descreve o pacote: nome, versão 0.3.3, Python mínimo, ficheiros web incluídos e comando quantumtrace. As dependências obrigatórias estão vazias. A biblioteca cryptography é opcional para certificados e chaves públicas, não para reconhecer chamadas em código.

LICENSE

Contém a licença MIT do código QuantumTrace distribuído. Explica as permissões de utilização e a ausência de garantias. Não substitui as condições aplicáveis aos dados fornecidos pela organização ou às ferramentas externas.

.gitignore

Indica ao Git quais os resultados, dados locais, ambientes e ficheiros compilados a ignorar. Não altera o scanner e não apaga nada. A pasta data pode estar no ZIP mesmo estando ignorada pelo Git.

8. Documentação da raiz: o contrato do protótipo

Estes documentos explicam a utilização e os limites. São texto para pessoas, não regras executadas pelo programa. A implementação das classificações está em policy.py.

README.md

Apresenta instalação, formatos, tipos de evidência, funcionamento local e esquema JSON. É o mapa geral do projeto. Distingue samples dos dados da organização e explica que o inventário não certifica sistemas completos.

QUICKSTART_PT.md

Guia de arranque em português para Windows, extração do ZIP, dependências e utilização da app. Ajuda a resolver problemas operacionais antes de interpretar resultados.

DEMO_PT.md

Guia anterior para demonstrar a solução e interpretar o relatório. Consulta-o como referência de utilização. Para a apresentação atual, usa o roteiro novo incluído neste pacote, com os números da versão 0.3.3.

POLICY.md

Documenta a política junctionx-2026-context-policy-v2, incluindo a extensão para hashes, a informação em falta e a fórmula do índice. Permite discutir as decisões sem fingir que os pesos ou o alvo teórico são categorias NIST.

VALIDATION.md

Regista verificações, ambiente de teste, resultados e limites. A versão atual regista 86 testes aprovados. As expectativas dos exemplos são manuais, sem etiquetas oficiais de precisão. É a base para responder ao júri sobre confiança.

APP_API.md

Documenta os pedidos JSON locais para iniciar análises, acompanhar tarefas, obter relatórios, comparar e apagar histórico. Explica autenticação de sessão, limites de upload e armazenamento SQLite. A nossa app já tem uma API JSON própria.

9. Motor Python: entradas, deteção e inferência

Estes ficheiros transformam bytes de entrada em observações. A deteção responde ao que aparece no ficheiro; a política responde a como classificar essa observação.

quantumtrace/__init__.py

Identifica o pacote e define __version__ como 0.3.3. Outros módulos usam esta versão nos relatórios e na interface. Não analisa ficheiros.

quantumtrace/__main__.py

É a porta de entrada de python -m quantumtrace. Encaminha a execução para cli.main. Existe para poderes iniciar a app ou uma análise sem instalar um comando global.

quantumtrace/cli.py

Interpreta os comandos doctor, scan e serve. doctor mostra dependências; scan chama o inventário e escreve HTML/JSON; serve inicia a app local. As opções strict e fail-on-critical permitem devolver códigos de saída úteis em automações.

quantumtrace/inventory.py

Percorre os ficheiros, rejeita ligações simbólicas, aplica limites e escolhe o detetor pela extensão. O Collector uniformiza cada observação, junta a avaliação e evita duplicações com a mesma identidade. make_report calcula contagens, cobertura e índice. build_priorities agrupa por ficheiro, recurso AWS ou sessão TLS e junta motivo, próximo passo e referências à evidência. A ordem considera o estado da evidência; não calcula impacto empresarial. É o coordenador do motor.

quantumtrace/scanners.py

Contém a inspeção de JSON, Python e certificados e encaminha Java/Go. Em JSON procura campos conhecidos. Em Python analisa a árvore sintática e chamadas suportadas, com imports e aliases. Não importa nem executa os programas recebidos. Certificados usam cryptography apenas se disponível.

quantumtrace/static_facts.py

Ajuda a inferir constantes e comprimentos de chaves Python. Por exemplo, token_bytes(16) dá um comprimento de 16 bytes, convertido em 128 bits. Segue alguns chamadores observados; valores reatribuídos, contraditórios ou fora da gramática suportada ficam desconhecidos. Não é análise semântica completa.

quantumtrace/source_patterns.py

Reconhece padrões Java/Go e infere alguns parâmetros dentro de funções. Mascara strings/comentários ao procurar chamadas, acompanha constantes, inicializações e imports Go conhecidos. Usa padrões limitados, não um compilador. Nomes Java ambíguos ou wrappers complexos podem causar omissões ou exigir revisão.

10. Motor Python: rede, política e aplicação

A separação em módulos permite melhorar um detetor sem reescrever a interface e rever a política sem alterar os ficheiros originais.

quantumtrace/tls.py

Lê eventos TLS normalizados, JSON do TShark ou capturas PCAP/PCAPNG através de TShark. Traduz identificadores suportados, guarda frame/stream/endpoints e distingue ClientHello oferecido de ServerHello selecionado. Regista componentes híbridos separadamente. Campos cifrados ou identificadores desconhecidos não são adivinhados.

quantumtrace/cloud.py

Interpreta campos AWS KMS e cinco nomes de políticas ELB documentados. Enriquece declarações com algoritmos, funções e capacidades permitidas. Não contacta uma conta AWS. Uma política permitida é configuração, não uma ligação negociada; nomes não mapeados ficam parciais.

quantumtrace/policy.py

parse_label normaliza nomes e decompõe suites ou grupos em primitivas. classify atribui nível, explicação, recomendação, referências e informação em falta. Não calcula o índice. Aqui estão as regras Shor, AES, PQC, hashes e as decisões conservadoras sobre HMAC/PBKDF2.

quantumtrace/runtime.py

Procura TShark no PATH e nas localizações Windows habituais. doctor usa diagnostics para informar Python, TShark e cryptography. false em cryptography significa componente opcional ausente; não impede analisar os códigos e JSON incluídos.

quantumtrace/report.py

Combina o inventário JSON com o modelo HTML, CSS e JavaScript num relatório autocontido. write_reports grava report.json e report.html. Escapa os dados ao inseri-los no HTML. O mesmo relatório serve a app e a consulta offline; não executa uma nova análise quando é aberto.

quantumtrace/app.py

Implementa o servidor HTTP local, API, fila com uma análise de cada vez, uploads temporários e histórico SQLite. Usa o mesmo scan_path do terminal. Compara relatórios por identidade de evidência e controla eliminação persistente. A porta fica em 127.0.0.1 e os pedidos usam um token de sessão. É o backend, não um serviço de produção na internet.

11. Interface: o que vês no navegador

HTML define a estrutura, CSS o aspeto e JavaScript as interações. Há uma interface da aplicação e outra do relatório para permitir exportar o resultado sem depender do servidor.

quantumtrace/web/index.html

Estrutura da app: nova análise, seleção de ficheiros/pasta, histórico, comparação e área do relatório. Tem rótulos PT/EN. Não contém o resultado fixo de uma análise.

quantumtrace/web/app.css

Estilos da app: tipografia, cores, distribuição das áreas e adaptação ao tamanho do ecrã. Alterar este ficheiro muda a experiência visual, não as classificações.

quantumtrace/web/app.js

Lê os ficheiros selecionados, prepara o pedido JSON, chama a API e acompanha a tarefa até terminar. Gere histórico, comparação, confirmações de eliminação, idioma e downloads. Conserva o token na sessão do separador e remove-o do URL visível.

quantumtrace/web/report.html

Modelo do relatório com espaços que report.py preenche. Define cartões, prioridades por ficheiro/recurso, sessões TLS, fila de revisão, tabela, cobertura e diálogo de evidência. A política tem uma explicação curta e detalhes expansíveis. É um template, diferente de reports/report.html já preenchido.

quantumtrace/web/report.css

Estilos específicos do relatório: níveis, tabelas, diálogo e disposição responsiva. Ajuda a ler o relatório na app ou num ficheiro offline.

quantumtrace/web/report.js

Lê o inventário embutido no relatório e apresenta filtros, pesquisa, páginas de 50 observações, páginas de seis grupos de prioridades, casos por rever, sessões e detalhes. As prioridades têm pesquisa, filtros por ação e acesso às observações de suporte. Mostra a recomendação e a regra de cada observação. Usa textContent para apresentar evidência e permite descarregar o JSON. Não redescobre a criptografia no navegador.

12. Testes: o que verificam e o que não provam

São 86 testes automatizados na distribuição atual. Estes testes são parte do desenvolvimento assistido por IA e não uma auditoria independente. As expectativas concretas permitem detetar regressões, sem medir todas as ocorrências que o scanner pode falhar.

tests/test_quantumtrace.py

31 testes do comportamento básico: regras AES/PQC, comentários e strings, chamadas conhecidas, configuração, TLS oferecido/negociado, formatos inválidos, ausência de TShark, certificados e incorporação segura de dados no HTML. Verificam decisões concretas do motor.

tests/test_extended_detection.py

19 testes dos padrões acrescentados: inferência por chamadores, valores contraditórios, aliases, Java/Go, HMAC/KDF, AWS e proveniência TLS. Evitam que melhorar a deteção transforme uma suposição em certeza.

tests/test_context_policy.py

20 testes: oito verificações de contexto e doze de prioridades, incluindo ofertas, híbridos, recursos, estados da evidência e compatibilidade com relatórios antigos. As verificações de contexto abrangem SHA-256 em usos diferentes, modelo conservador TLS, ChaCha20 com tamanho conhecido, lacuna OAEP e distinção entre tamanho da saída PBKDF2 e entropia da palavra-passe. Verificam a política, não um ataque quântico real.

tests/test_starter_dataset.py

4 testes de integração com o conjunto da organização. Conferem famílias esperadas nos 24 ficheiros de código, seis casos AES, KMS/ELB e os 97 pares TLS com sete grupos híbridos. Comparar conjuntos de famílias é mais limitado do que medir todas as ocorrências.

tests/test_app.py

12 testes do serviço HTTP real: uploads passivos, persistência, comparação, sessão/origem, rejeição de entradas inválidas, eliminação e bloqueio durante análises. Testam o backend. Não equivalem a uma verificação visual de todas as ações no Windows.

13. Dados da organização: documentação e metadados

Os 35 ficheiros em data vieram do repositório fornecido. Foram conservados sem alteração no snapshot indicado no fim do guia. Sete são documentação/metadados fora dos formatos de deteção; isto explica 35 entradas e 28 ficheiros analisados.

data/README.md

Mapa dos três tipos de dados fornecidos: código, configuração cloud e captura. É documentação, não um resultado nem um ficheiro de código analisável pelo motor.

data/.gitignore

Regras originais do conjunto para ignorar ambientes, caches e ficheiros Java compilados. O scanner reporta-o como formato não suportado, com zero observações.

data/cloud-posture/README.md

Explica que os exports AWS são sintéticos e que os identificadores são exemplos. Evita apresentar os dados como uma conta empresarial real.

data/network-capture/README.md

Descreve uma captura sintética de 2000 pacotes, com vários protocolos e aproximadamente 135 sessões de rede. O QuantumTrace só extrai os campos TLS suportados; as 97 sessões do relatório não abrangem todo esse tráfego.

data/sample-repos/README.md

Descreve os programas Python/Java/Go para inspeção. O original inclui instruções para os executar, mas o QuantumTrace não precisa disso: lê o código passivamente.

data/sample-repos/go/go.mod

Metadados do módulo Go e versão da linguagem. Não é uma chamada criptográfica e esta versão não inventaria dependências a partir dele.

data/sample-repos/python/requirements.txt

Declara cryptography==46.0.5 para executar os exemplos originais Python. Não é uma dependência obrigatória do nosso scanner e não é analisado como inventário de bibliotecas nesta versão.

14. Dados da organização: JSON e captura

Um ficheiro pode gerar várias observações. Uma suite TLS pode gerar AES, hash, assinatura e troca de chaves separados. As contagens abaixo são as do relatório incluído, não recursos únicos.

data/cloud-posture/acm_certificates.json

Export sintético de metadados ACM. Declara algoritmos e assinaturas dos certificados, dando 15 observações. O motor lê campos reconhecidos do JSON; não precisa de cryptography porque não está a descodificar um certificado PEM/DER.

data/cloud-posture/kms_key_inventory.json

Export de 10 chaves KMS com KeySpec, utilização e algoritmos. Gera 26 observações. A região permite interpretar SYMMETRIC_DEFAULT nos casos mapeados. A componente SHA-256 de RSA-OAEP fica por rever por falta de uma regra específica da construção.

data/cloud-posture/load_balancer_listeners.json

Declara listeners e políticas TLS. cloud.py expande cinco nomes reconhecidos em capacidades, dando 230 observações configuradas. Uma mesma localização pode representar várias suites permitidas. Isto não prova que essas suites tenham sido utilizadas.

data/network-capture/network.pcap

Captura sintética lida por TShark, que gera 1253 observações: 955 opções oferecidas e 298 componentes selecionados. O relatório contém 97 sessões com ServerHello, sete com grupo X25519MLKEM768. Não significa que todas as comunicações ou a autenticação estejam totalmente avaliadas.

15. Dados da organização: oito ficheiros Python

Estes exemplos da organização incluem lógica de aplicação além das chamadas criptográficas. O scanner inspeciona os padrões suportados sem executar os programas. A contagem é o número de observações deste relatório, não de vulnerabilidades.

data/sample-repos/python/artifact_manifest.py

Calcula checksum SHA-384 e tags HMAC-SHA-256 para um manifesto. Resultado: 2 observações, SHA-384 resiliente pela política e HMAC por rever. O nome do digest do HMAC não demonstra a entropia da chave nem a construção completa.

data/sample-repos/python/legacy_accounts.py

Usa MD5 diretamente para representar palavras-passe. Resultado: 1 observação média, com preocupação clássica adicional. O nível do desafio não substitui uma avaliação de armazenamento de palavras-passe.

data/sample-repos/python/license_token.py

Gera RSA de 1024 bits e usa SHA-1 ao assinar/verificar licenças. Resultado: 3 observações, RSA crítico e duas chamadas SHA-1 médias. O scanner não resolve genericamente os métodos sign/verify de qualquer objeto de chave Python.

data/sample-repos/python/member_credentials.py

Deriva e verifica credenciais com PBKDF2-HMAC-SHA-256, 600000 iterações e saída de 32 bytes. Resultado: 4 observações, duas KDF por rever e duas componentes hash resilientes no contexto suportado. A saída de 256 bits não prova uma palavra-passe com 256 bits de entropia.

data/sample-repos/python/report_export.py

Cifra com AES/ECB e chave literal de 16 bytes. Resultado: AES-128, médio, 1 observação. Reconhecer o tamanho não resolve os problemas de uma chave fixa, ECB ou geração de tokens. Esta versão não descobre automaticamente todas as falhas clássicas do programa.

data/sample-repos/python/secure_transfer.py

Gera uma chave de 32 bytes e usa AESGCM para cifrar/decifrar. O motor segue as chamadas observadas para inferir AES-256. Resultado: 2 observações resilientes. Nonces, chaves, caminhos executados e implantação continuam a precisar de revisão própria.

data/sample-repos/python/update_feed.py

Calcula/verifica um checksum SHA-1 de um pacote. Resultado: 1 observação média. Um checksum não autenticado não demonstra quem publicou os dados; o inventário não substitui o desenho de distribuição segura.

data/sample-repos/python/webhook_signature.py

Gera uma chave Ed25519 e assina/verifica mensagens de webhook. Resultado: 1 observação crítica na geração de chave. Ed25519 é uma assinatura clássica e não uma norma PQC. Os métodos do objeto não são todos resolvidos por este detetor.

16. Dados da organização: oito ficheiros Java

Estes exemplos da organização incluem lógica de aplicação além das chamadas criptográficas. O scanner inspeciona os padrões suportados sem executar os programas. A contagem é o número de observações deste relatório, não de vulnerabilidades.

data/sample-repos/java/BatchExporter.java

Usa AES/ECB/PKCS5Padding e chave literal de 16 bytes. Resultado: 1 observação AES-128 média. O padrão Java também acrescenta um aviso clássico sobre ECB. O inventário não é uma auditoria completa do gerador de tokens ou da gestão de chaves.

data/sample-repos/java/ConfigSync.java

Usa MessageDigest SHA-384 e Mac HmacSHA256 para configurações. Resultado: 2 observações, uma resiliente e uma por rever. Identificar o algoritmo do MAC não resolve os parâmetros do segredo.

data/sample-repos/java/CustomerDirectory.java

Calcula MD5 para palavras-passe. Resultado: 1 observação média com aviso de criptografia depreciada. O caso mostra deteção em Java sem compilar o programa.

data/sample-repos/java/DeviceAttestation.java

Usa KeyPairGenerator e Signature Ed25519 para atestação. Resultado: 3 observações críticas. As chamadas em locais distintos são observações distintas; não são três dispositivos únicos.

data/sample-repos/java/IdentityVault.java

Usa PBEKeySpec e SecretKeyFactory PBKDF2WithHmacSHA256, com 600000 iterações e saída de 256 bits. Resultado: 1 observação por rever. O motor segue o spec passado à factory, mas a política não converte o comprimento da saída em entropia da palavra-passe.

data/sample-repos/java/PatchFeed.java

Usa MessageDigest SHA-1 para checksum de atualizações. Resultado: 1 observação média. O scanner extrai a chamada getInstance e a etiqueta conhecida.

data/sample-repos/java/ReceiptSigner.java

Gera RSA-1024 e pede Signature SHA1withRSA para assinar/verificar recibos. Resultado: 5 observações, três RSA críticas e duas SHA-1 médias. A etiqueta composta é decomposta em componentes para não esconder o hash.

data/sample-repos/java/SessionCipher.java

Usa geração de chave AES de 256 bits e Cipher AES/GCM/NoPadding. Resultado: 3 observações resilientes. Geração, cifragem e decifragem são locais diferentes; isso não certifica o tratamento de nonces ou toda a sessão.

17. Dados da organização: oito programas Go

Estes exemplos da organização incluem lógica de aplicação além das chamadas criptográficas. O scanner inspeciona os padrões suportados sem executar os programas. A contagem é o número de observações deste relatório, não de vulnerabilidades.

data/sample-repos/go/agent-enrollment/main.go

Gera chaves e assina/verifica dados com crypto/ed25519. Resultado: 4 observações críticas. Uma chamada de verificação para uma mensagem alterada também é uma ocorrência de código, independentemente de ser apenas um teste no programa.

data/sample-repos/go/auth-gateway/main.go

Implementa derivação de palavras-passe com um ciclo de HMAC-SHA-256. Resultado: 2 observações HMAC por rever. O scanner reconhece chamadas HMAC, mas não demonstra que um ciclo arbitrário implementa corretamente PBKDF2.

data/sample-repos/go/batch-pipeline/main.go

Cifra blocos com AES e chave literal de 16 bytes. Resultado: 1 observação AES-128 média. A inspeção não verifica automaticamente todas as propriedades do modo de operação ou dos tokens gerados com math/rand.

data/sample-repos/go/config-sync/main.go

Usa SHA-384 e HMAC-SHA-256 para checksum e tag. Resultado: 2 observações, SHA-384 resiliente e HMAC por rever. Mostra a distinção entre hash público e autenticação com segredo.

data/sample-repos/go/invoice-service/main.go

Gera RSA-1024 e usa RSA PKCS1v15 com SHA-1 em assinaturas. Resultado: 5 observações, três RSA críticas e duas SHA-1 médias. O tamanho pequeno acrescenta uma preocupação clássica ao risco quântico da família.

data/sample-repos/go/member-directory/main.go

Usa crypto/md5 para representar palavras-passe. Resultado: 1 observação média com preocupação clássica. A lógica de comparação de palavras-passe não muda a classificação da primitiva.

data/sample-repos/go/release-feed/main.go

Usa crypto/sha1 para checksum de pacotes. Resultado: 1 observação média. A deteção baseia-se no pacote e na chamada, não no nome release-feed.

data/sample-repos/go/telemetry-relay/main.go

Gera uma chave de 32 bytes e usa AES/GCM para cifrar/decifrar. Resultado: 2 observações AES-256 resilientes através de chamadas observadas. O algoritmo, o modo e a correção do sistema são avaliações diferentes.

18. Exemplos próprios em samples

São cinco fixtures sintéticas criadas durante o desenvolvimento. Exercitam comportamentos pequenos, incluindo incerteza e etiquetas compostas. Não vieram da organização e não medem precisão independente.

samples/application.py

Contém RSA-2048, AES com tamanho desconhecido, AES com literal de 32 bytes e SHA-1. Inclui nomes em comentário/string como controlo contra falsos positivos. A chave nula demonstrativa não é uma recomendação de geração segura.

samples/Application.java

Tem Cipher AES/GCM sem tamanho de chave, KeyPairGenerator RSA e MessageDigest SHA-1. Demonstra que identificar AES não permite inventar 256 bits quando a inicialização da chave não aparece.

samples/application.go

Tem crypto/rsa com geração de 2048 bits e crypto/sha1. É um caso pequeno para confirmar os padrões Go e as classificações correspondentes.

samples/cloud.json

Mistura campos reconhecidos, AES com e sem tamanho, PQC final e nomes legados. Também inclui suites TLS e texto descritivo. Testa normalização e decomposição sem tratar toda a prosa como evidência.

samples/tls.json

Define eventos no formato quantumtrace.tls/1.0. Inclui ClientHello e ServerHello, grupo clássico e grupo híbrido. Permite exercitar a lógica TLS sem necessitar de TShark. Não é uma captura real.

19. Relatórios e preparação já existentes

Estes ficheiros explicam ou conservam análises. Um relatório preenchido é uma saída gerada, enquanto um validador é código que chama o mesmo motor com expectativas separadas.

reports/report.html

Relatório offline previamente gerado para os dados da organização. Inclui os dados, estilos e interações num ficheiro. Abrir não analisa novamente. ANALISAR_DADOS.cmd pode substituí-lo com uma nova análise.

reports/report.json

Inventário estruturado correspondente, com schema, versão, política, resumo, ficheiros, observações e avisos. A app não o importa automaticamente para o histórico. É um esquema CBOM próprio, não CycloneDX validado.

Preparacao/COMECA_AQUI.md

Guia anterior de preparação, expectativas dos oito casos e tarefas recomendadas. Inclui a alteração AES de 16 para 32 bytes. Algumas frases sobre confirmação no Windows precedem a tua captura com os resultados esperados.

Preparacao/GUIAO_E_PERGUNTAS.md

Guião provisório anterior com perguntas prováveis. A contagem foi atualizada para 86 testes. Usa o novo roteiro inglês como versão principal da apresentação.

Preparacao/QuantumTrace_Proximos_Passos.pdf

Plano de preparação com uma nova primeira página a identificar as melhorias executadas em 0.3.3. As páginas seguintes conservam o plano anterior como referência histórica. É documentação estática e não um programa. Não era um PowerPoint.

Preparacao/VALIDAR_EXEMPLOS.cmd

Atalho Windows que encontra Python, executa validar_exemplos.py e abre Preparacao/resultados/report.html. Precisa de a pasta Preparacao estar junto de INICIAR_APP.cmd, como já acontece no ZIP atual.

Preparacao/validar_exemplos.py

Analisa os oito ficheiros com o motor do projeto e compara algoritmo, tamanho, nível e tipo de evidência com expectativas manuais. Verifica também o conjunto de ficheiros e os totais. Gera um relatório e validacao.json. Não executa os exemplos.

Preparacao/resultados_esperados.json

Define os resultados manuais esperados antes da validação: nove observações, com dois críticos, dois médios, quatro resilientes e um desconhecido. Não é o resultado calculado e não deve ser alterado só para forçar um teste a passar.

Preparacao/resultados/report.html

Relatório offline gerado para os oito exemplos, com índice 37,5. VALIDAR_EXEMPLOS.cmd atualiza-o. Não é o relatório dos dados da organização.

Preparacao/resultados/report.json

Inventário JSON dos oito exemplos. Serve para conferir as observações em detalhe e reproduzir a conta do índice. Pode ter nova data quando repetes a validação.

Preparacao/resultados/validacao.json

Registo gerado da comparação: esperado, obtido, aprovação de cada caso e hash do ficheiro de expectativas. É uma verificação de comportamentos concretos, sem medir precisão geral.

Preparacao/resultados/verificacao_app.json

Registo atualizado em 0.3.3 de uma verificação pelo serviço HTTP em Linux: oito casos, mudança de comprimento, comparação, histórico e exportação. Explicita que não é uma verificação nativa do launcher Windows ou de toda a aparência no navegador.

20. Os oito ficheiros da demonstração

São exemplos próprios, adicionais ao desafio. Podes selecionar os oito de uma vez. Analisa apenas esta subpasta, não toda a pasta Preparacao, para não misturar programas de validação e resultados com os dados de demonstração.

Preparacao/FICHEIROS_NOVOS/01_aes128.py

token_bytes(16) e AESGCM: 16 bytes vezes 8 dá 128 bits. Esperado: AES-128 médio, uma observação. É o ficheiro a editar para demonstrar uma nova análise.

Preparacao/FICHEIROS_NOVOS/02_aes256.py

Importa token_bytes com o alias criar_chave e pede 32 bytes. Esperado: AES-256 resiliente, uma observação. Demonstra o tamanho e resolução de um alias conhecido em Python.

Preparacao/FICHEIROS_NOVOS/03_rsa.py

Importa rsa como gerador e gera uma chave de 3072 bits. Esperado: RSA crítico, uma observação. Aumentar o módulo não elimina a exposição teórica da família a Shor.

Preparacao/FICHEIROS_NOVOS/04_aes_desconhecido.py

Define preparar_cifra(chave) e chama AESGCM(chave) na linha 6. Não mostra chamadores nem origem/comprimento da chave. Esperado: AES por rever. A recomendação é determinar 128, 192 ou 256 bits, não adivinhar.

Preparacao/FICHEIROS_NOVOS/05_sem_chamadas_crypto.py

Contém uma chamada RSA em comentário e nomes AES/RSA numa string, mas só soma preços. Esperado: zero observações. É um controlo negativo construído; zero noutro projeto não prova ausência de criptografia.

Preparacao/FICHEIROS_NOVOS/06_ChaveJava.java

Pede KeyGenerator AES e inicializa o gerador a 256 bits. Esperado: uma observação resiliente. Mostra um padrão Java sem compilar nem executar.

Preparacao/FICHEIROS_NOVOS/07_chave_go.go

Cria um slice de 16 bytes, usa crypto/rand e chama aes.NewCipher. Esperado: AES-128 médio, uma observação. O motor segue o comprimento suportado, sem executar rand.Read.

Preparacao/FICHEIROS_NOVOS/08_config_nova.json

Declara três serviços: assinatura RSA-3072, AES-256 e ML-KEM-768. Esperado: três observações configuradas, uma crítica e duas resilientes. 768 é um nome de conjunto de parâmetros ML-KEM, não o tamanho de um segredo em bits.

21. O que tens de conseguir explicar ao júri

Escolhe três casos que compreendas: um RSA crítico, um AES com tamanho conhecido e um AES por rever. Para cada um, mostra o ficheiro, a linha/campo, o parâmetro, a regra e o próximo passo. Não tens de explicar todas as 1574 observações nem decorar cada ficheiro do motor.

Sobre confiança: o conjunto original foi preservado, cada observação tem origem inspecionável e há 86 testes aprovados com expectativas específicas. O kit adicional confirma oito casos novos, incluindo um negativo. Isto sustenta comportamentos concretos. Não existem etiquetas oficiais completas nem uma medição independente de precisão/recall em projetos empresariais.

Sobre utilização real: um programador pode selecionar uma pasta de código e obter um inventário inicial dos formatos e APIs suportados. Depois revê casos desconhecidos e liga as observações aos recursos reais. Falta ampliar detetores, validar independentemente, mapear dependências e priorizar por criticidade, exposição e duração de confidencialidade.

Sobre recomendações: cada caso tem Next step. RSA pede um plano PQC adequado à função; AES-128 pede revisão de política e eventual AES-256; um caso desconhecido pede a informação em falta. A app não corrige código automaticamente. O resumo de prioridades já

agrupa estas decisões por ficheiro ou recurso e distingue o estado da evidência. A prioridade empresarial continua dependente de contexto externo.

Sobre o site: é uma app web local, com servidor Python e interface no navegador. 127.0.0.1 é o endereço deste computador, não o teu IP público. O token de sessão é separado e deve manter-se privado. O relatório exportado abre como ficheiro offline e não precisa da app em execução. Links externos de documentação precisam de internet.

Sobre IA: explica que usaste IA para acelerar implementação, testes e documentação. Na análise desta versão não existe um agente LLM a decidir classes, nem chamadas pagas. O motor usa regras explícitas; a API local transporta dados e SQLite guarda relatórios. JSON, API, IA e base de dados são conceitos diferentes.

O que não deves prometer: análise de todos os binários/linguagens, descoberta exaustiva, execução real de cada chamada, precisão de 98,5%, segurança de toda a sessão por existir AES-256/PQC, CycloneDX validado ou correção automática de vulnerabilidades.

22. Materiais novos e ordem de estudo

Este pacote de apresentação acrescenta o presente guia, um PowerPoint em inglês com cinco slides principais e três de apoio, uma cópia dos slides em PDF, e um roteiro completo com ações, falas, alternativa de três minutos e respostas ao júri. Os slides de apoio ficam ocultos na apresentação normal; consulta-os durante perguntas.

1. Lê as secções 1 a 6 deste guia e reproduz a conta de 37,5 sem notas.
2. Abre os seis ficheiros centrais sugeridos na secção 1 e encontra as funções descritas.
3. Na app, analisa os oito exemplos e explica o 04 sem inventar o tamanho.
4. Ensaia a mudança no 01 de token_bytes(16) para token_bytes(32). Guarda e volta a selecionar o ficheiro antes da nova análise. Restaura 16 no fim.
5. Lê o roteiro inglês em voz alta e cronometra. Os tempos são uma proposta de cinco minutos, com cortes para três; confirma o tempo permitido pela organização.
6. Antes de apresentar, verifica a app e o relatório offline no teu PC. A verificação de ficheiros nesta preparação não substitui esse ensaio no PowerPoint e no navegador da apresentação.

23. Fontes e identificação da versão

Base principal: código, testes e relatórios da distribuição QuantumTrace 0.3.3 inspecionados nesta preparação; enunciado do desafio transcrito na conversa. As contagens pertencem aos relatórios incluídos. A revisão final conservou as regras de classificação e os dados. Melhorou recomendações para híbridos, prioridades e apresentação da política; foram executados 86 testes e verificações HTTP/DOM.

Dados fornecidos: https://github.com/joaoalves3901/JunctionX-QuantumTrace

Snapshot conservado: 39e8f18dcfb5edc66eda0f47bb44884aa7db5c9c.

Normas e modelos referenciados na política:

- NIST PQC FAQ: https://csrc.nist.gov/projects/post-quantum-cryptography/faqs
- NIST FIPS 203, ML-KEM: https://csrc.nist.gov/pubs/fips/203/final
- NIST FIPS 204, ML-DSA: https://csrc.nist.gov/pubs/fips/204/final
- NIST FIPS 205, SLH-DSA: https://csrc.nist.gov/pubs/fips/205/final
- Modelo de colisões: https://arxiv.org/abs/quant-ph/9705002
- Modelo de procura: https://arxiv.org/abs/quant-ph/9605034
- ChaCha20/Poly1305: https://www.rfc-editor.org/rfc/rfc8439.html
- PBKDF2: https://www.rfc-editor.org/rfc/rfc8018.html
- HMAC: https://www.rfc-editor.org/rfc/rfc2104

As fontes apoiam mecanismos e parâmetros. Os pesos do índice e a extensão de níveis para hashes são escolhas do projeto. FAQ NIST e página FIPS 203 consultadas em 10 de outubro de 2026. O guia não é uma revisão de segurança independente.